

Agent 内核拆解

—— 110 行的 while 循环，到工业级差在哪

六周拆解 pi / codex / grok-build 三个工业级 coding agent 的收官系列：每篇一篇解析笔记 + 一个可运行的 mini-agent 阶段代码，从"Agent 就是个 while 循环"一路拆到"沙箱与审批是两件独立的事"

样本仓库：github.com/yanhua1010/build-your-own-coding-agent (41★, HEAD 1e607db, 六篇全发布) | 对照上游：earendil-works/pi (~78k★) · openai/codex (~101k★) · xai-org/grok-build (~23k★) | 实验：mini-agent 真跑通 + 协议 shim 自写 + 136 条离线断言全过 | 产出：2026-10-03

起点

"pi 这种也算工业级？
加几个工具不就完了？"
系列因这个质疑而生

做法

拆三个工业级实现
每篇笔记带行号引用
+ 一个可运行 step

结论

pi 算工业级，但不能拿来就用
内核可以 110 行
工业级差距是一张清单

疑问即目录 · 十二问拆完一个 Agent 内核

Q1 这个系列是什么来头？"Agent 就是个 while 循环"这句话可信吗？

出处是 X 用户 @yanhua1010 的六周系列【Agent 内核拆解】，收官帖回应读者的质疑——"pi 这种也算工业级？加几个工具不就完了？"。配套 GitHub 仓库 **build-your-own-coding-agent** 把方法论写得很清楚：**每篇文章 = 一篇带行号引用的源码解析笔记（对照 pi 的真实 commit）+ 一个能独立运行的 mini-agent 阶段代码**。五篇笔记覆盖：①agent loop ②统一 LLM API 与错误契约 ③工具调用 ④上下文压缩 ⑤权限与安全，第六篇收官总结。

"while 循环"这句话**可信且有实锤**：pi 的整个内核 `agent-loop.ts` 只有 792 行，其中主循环本体 `runLoop` 只占 126 行；教学版 mini-agent 的第一步 (step01) **118 行就写完了一个能真干活的 coding agent**。但"能跑"和"工业级"之间的距离，正是这个系列六周要讲清的东西。



mini-agent 路线图：每篇一个可运行阶段，逐步逼近工业级

Q2

118 行的 mini-agent 里到底有什么？"智能"藏在哪一行？

step01 的 `agent.mjs` 一共四块：三个工具（`read_file` / `write_file` / `run_command`，各配 schema 和执行函数）、一个最简沙箱（`safePath`：只准操作当前工作目录）、一个 `fetch`（调 OpenAI 兼容接口）、核心循环。所谓"智能"就是这十几行：

```
// — 核心: agent loop。整个"智能"就在这个循环里 —
while (true) {
  const msg = await chat(messages);           // 1. 把对话发给模型
  messages.push(msg);
  if (!msg.tool_calls?.length) return;        // 2. 模型不再要工具 → 结束
  for (const call of msg.tool_calls) {
    try { result = String(tools[name].run(args)); }
    catch (err) { result = `错误: ${err.message}`; } // 3. 错误不抛出！
    messages.push({ role: "tool", tool_call_id: call.id, content: result }); // 4. 塞回对话
  }
}
```

已经学过 [[Agent循环]] 的会认出来：这就是那个循环的最裸形态——**没有任何框架魔法，调模型 → 有工具调用就执行 → 结果塞回消息列表 → 再调，直到模型收手**。真正值得划重点的是第 3 行注释：

关键设计 **工具执行出错时把错误文本"还给模型"而不是抛给用户**——模型读到"文件不存在"会自己 `ls` 排查再重试。Agent 的健壮性来自"把错误还给模型"，这就是自我纠错循环的最小实现。实验 B 真机验证过这一条（见实验部分）。

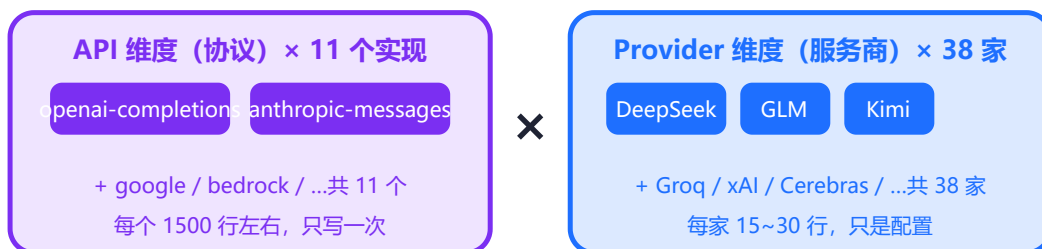
Q3

38 家模型厂商，为什么只需要 11 个 API 实现？

大多数人做"多模型支持"会按**服务商**建目录：`openai/`、`anthropic/`、`deepseek/`.....每加一家就重写一遍流式解析。pi 把它拆成了**两个独立的维度**：

- **API** = 协议格式（怎么说话）：11 种，如 `openai-completions`、`anthropic-messages`、`google-generative-ai` ...
- **Provider** = 服务商（跟谁说话）：38 家，每家只是一个 15~30 行的**纯配置文件**（`baseUrl` + 鉴权 + 模型清单 + 指向哪个 API）

DeepSeek、Groq、月之暗面、智谱、xAI.....20 家都兼容 OpenAI 的 `completions` 格式，**共用同一份 1504 行的解析代码**。接一家新 provider 的代价是 **15 行**（DeepSeek 的 provider 文件全文就 15 行，全是配置、零解析逻辑）。mini-agent 的 step02 照抄了这个设计：三家 provider 配置共用同一份 `openai-completions.mjs`。



协议 × 服务商两维拆分：新厂商只是加一行配置，不再重写解析

Q4

什么叫“永不 reject 的错误契约”？为什么上层敢一行 try/catch 都不写？

这是整个系列里 **最值得学的一条软件设计**。pi 的内核循环分支用的是 `stopReason` 字段，而不是 try/catch——凭什么？答案是一条**类型级契约**：`stream()` 函数的返回类型不是 `Promise`，而是同步返回一个 `EventStream`。**调用方无处 await，也就无处 try/catch——契约不靠文档约定，靠类型系统强制**。所有失败（鉴权挂了、网络断了、模块加载失败）都被编码成流内的 `error` 事件和一条 `stopReason: "error"` 的消息。妙处在于配套的第二条规则（note 03）：方向完全相反——**调模型的函数永不抛异常，但工具函数必须抛异常**。两条规则看似矛盾，其实看“错误的接收方是谁”就通了：

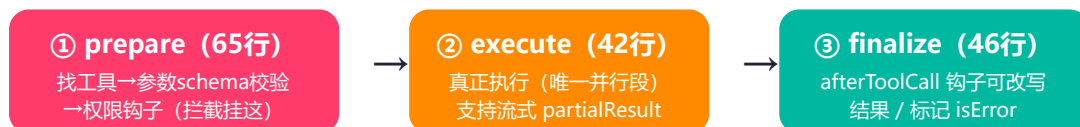
	模型调用失败	工具执行失败
接收方	程序（处理不了网络故障，只能优雅退出）	模型（读懂“文件不存在”会自己排查重试）
处理方式	降级成 <code>stopReason</code> 事件，绝不 reject	抛出异常 → 内核 catch 成 <code>isError: true</code> 结果回喂

一眼看懂 判断一条错误该“抛”还是该“吞”，先问：**这个错误的读者是程序还是模型？**读者是模型，就用模型读得懂的话（错误文本）说给它听。

Q5

一次工具调用在内核里经过哪三段？为什么一半的内核代码都在管工具？

反直觉的数字：pi 内核 792 行里，**工具执行相关占 418 行 (53%)**，主循环本体只有 126 行。模型那边只有一次 `stream()` 调用，工具这边才是内核真正的复杂度。一次调用经过**三段式流水线**：



三段式：权限审批挂在 prepare（钩子返回 {block:true} 即拒绝，拦截信息回喂模型）

mini-agent 的 step03 抄了这套骨架，还加了三个防御，每个都是“只有真踩过坑才会写”的代码：

- **截断的整批作废**：输出被 token 上限截断时，工具参数 JSON 可能残缺却“看起来合法”、schema 校验都能通过——pi 的做法是**整批标记失败，一个都不执行**；
- **一票降级、全票终止**：一批工具里只要有一个声明必须串行（比如 edit 和 bash 混批），整批降级串行；反过来必须全批都要求终止才提前终止，单个工具不能替整批做主；
- **按文件排队**：并发写同一文件会互相踩，所以按文件路径排队、不同文件才并行。

Q6

并行执行三个工具，"完成顺序"和"喂回顺序"为什么必须不一样？

模型一次发出三个调用（耗时 3s / 1s / 2s）。如果串行执行要 6 秒；并行执行 3 秒就完。但这里藏着一个新手必踩的坑：**喂回模型的结果顺序必须保持模型给出的调用顺序（1-2-3），而完成顺序是乱的（2-3-1）**——因为对话历史必须确定，不然模型看到的上下文每次都不一样。实验 C 实测：

```
模型给出的调用顺序: call_1(3s) call_2(1s) call_3(2s)
实时事件（完成顺序）:
+0.0s 开始 call_1 +0.0s 开始 call_2 +0.0s 开始 call_3
+1.0s 完成 call_2 ← 谁快谁先 +2.0s 完成 call_3 +3.0s 完成 call_1
喂回模型的 toolResult 顺序: call_1 call_2 call_3 ← 历史还是 1-2-3
总耗时 3.0s（串行的话是 6s）
```

pi 更进一步，把"三个顺序"刻意错开：准备阶段**串行**（权限确认弹窗不能几个同时弹）、开始事件和结果消息按**源顺序**发（transcript 稳定）、完成事件按**完成顺序**发（UI 先亮先完成的）。**只把 execute 那一段包进 Promise.all——"只并行该并行的那一段"**，这是 mini-agent 做并行工具最容易做错的地方。

上下文快满了怎么办？compaction 的阈值、切割点和摘要模板是怎么设计的？

已经学过 [[上下文预算与战略压缩]]（Claude Code 用户侧的"什么时候主动 /compact"），这一篇补上**实现者视角**：agent 内部怎么自动做压缩（[[Compaction 上下文压缩算法]]）。pi 的方案四步：

- **算账**：token 估算用"字符数÷4"，且**优先用 provider 返回的真实 usage，只对之后新增的消息做估算**；
- **触发**：两种路径——阈值触发（用到 `window - reserveTokens` 就压，默认预留 16384）和溢出触发（provider 报错，用约 25 种正则匹配各家溢出报错文案，还要排除限流误判）；
- **切割**：从后往前累加，保留最近 `keepRecentTokens`（默认 20000）原文，更早的进摘要；切割点只落在"整轮边界"，落在轮中间要做 split-turn 两段摘要；
- **摘要**：固定模板（Goal / Progress(Done/In Progress/Blocked) / Key Decisions / Next Steps），迭代压缩时带旧摘要增量更新；末尾还附**文件操作清单**（读过/改过哪些文件）——**从 tool_calls 参数里提取，不靠模型记忆**。

压缩前 799 tok

✂ 切割点=第7条



压缩后 594 tok（释放 205）



实验 D 实测数字：800 窗口、阈值 680，799 tok 触发；切割点第 7 条，799→594

四个 coding agent 的安全哲学差在哪？"沙箱"和"审批"为什么是两件独立的事？

收官系列最核心的认知框架：**两个正交的轴——sandbox（能不能做到：OS 层面 agent 碰得到什么）× approval（该不该做：这个动作要不要人点头）**。DeepSeek Harness 的文档把话说得最透：“审批对话框不是文件系统边界，沙箱也不能替代用户意图。”一个 agent 可以有沙箱没审批，也可以有审批没沙箱。四方实测对照：

维度	pi	DeepSeek Harness	codex	grok-build
OS 沙箱	无（故意，外包装容器）	内建，默认 read-only（开）	内建，默认开（Seatbelt/bwrap+seccomp）	内建（Landlock/Seatbelt），默认 off（关）
文件隔离	不硬隔离	写限 cwd	内核硬隔离，.git 强制只读	worktree 独立副本（CoW）
审批	钩子，默认不拦	插件化，headless fail-closed	4 档，默认 OnRequest	5 档，默认 Ask，Auto 档走 LLM 分类器
网络	外包	沙箱不管网络	默认禁网 + 域名代理	子进程 seccomp 仅 strict/Linux
独门	诚实的最小主义	沙箱/审批显式分离	.git 保护 + 升级审批	规则 DSL + 命令语义解析

实用外卖 “我用的 agent 安全吗”的答案常常取决于**默认配置和你跑在哪**，而不是它有没有这个能力。**别假设你的 agent 替你兜底，去看它默认跑在哪一档。**

grok-build 能力最强，为什么反而默认最不设防？"能力 ≠ 默认姿态"怎么理解？

系列里最大的反转：grok-build 把安全能力堆得最满——OS 沙箱（Landlock/Seatbelt+seccomp 手写 BPF、内核强制不可逆）、worktree 物理副本隔离、密钥脱敏、企业签名策略闹.....**但 OS 沙箱的默认值是 "off"**（解析顺序 requirement > CLI > env > config > "off"）。能力清单和默认姿态是两张表。
默认就把 agent 关进内核沙箱的只有 **codex 和 dsh**。还有一个耐人寻味的细节：codex 提供一键全放开 **--yolo**，官方注释明说**"仅供本身已在外部沙箱里运行的环境"**——连最重装的 codex 都承认"外部容器 + 关内建沙箱"是合法用法，pi 只是把这个默认前提摆到了台面上。

光谱 pi（全外包，诚实告诉你边界）—— grok-build（能力全但默认关，偏"外包心态"）—— codex / dsh（默认内建强制）。没有标准答案，只有显式做出的取舍。

pi 故意不做沙箱，是懒惰还是设计？

是设计，且声明得非常诚实。`SECURITY.md` 原话："the Pi coding agent intentionally does not have a sandbox"。它的信任模型只有一句话：**信任边界 = 运行 pi 的那个用户账号**。三条推论：①不防御"已拿到本地写权限的攻击者"（他本来就能改你的 shell 启动脚本）；②明说 prompt injection 防不住（`AGENTS.md` / 代码注释里塞一句恶意指令就能骗过它，官方用词 "cannot be protected against"，参见 [[提示注入]]）；③真正的隔离（容器/VM）是**用户的责任**。

但 pi 也不是完全裸奔，它把力气花在唯一真正的安全闸上：**工作区信任门控**——项目目录里凡是能"往 agent 里塞指令或代码"的资源（settings、extensions、skills、SYSTEM.md...）加载前都要先过信任检查，不信任就忽略并警告；且用"两遍加载"保证**在你做出信任决定之前，项目本地的扩展代码一行都不会被执行**。其余层面（审批钩子、路径解析、bash 工具的 env 清洗 + 进程树 kill）都是"降低误伤的护栏"而非安全边界。

从 110 行到工业级，差距清单到底有哪些？

收官帖承诺的总结。按笔记提炼，差距分四层：

层	mini-agent 已有	工业级还差什么
循环	单层 while	双层循环（内层管工具+用户插话 steering，外层管 follow-up 续跑）；失败路径也补发完整事件序列（事件序列在任何路径下闭合，UI 和持久化才能写得极简）
工具	try/catch + 顺序执行	三段式流水线 + 权限钩子 + 并行三顺序 + 截断整批作废 + 按文件排队
上下文	无限增长	token 记账 + 双路径触发压缩 + 切割点算法 + 结构化摘要 + 文件操作追踪
安全	safePath + env 清洗	信任门控 + 审批策略 + OS 沙箱（或诚实外包）+ 密钥防泄

笔记给的"值得抄的清单"优先级：①双层循环骨架 + emit 事件 ②错误编码进流、绝不 reject ③工具三段式 + 权限钩子 ④截断整批失败防御（一行 if，但显专业）⑤steering 队列 ⑥失败路径补发事件。**可以不抄的**：并行三顺序错开、双轨 context、运行中换模型——作为"工业级 vs 教学级"的对比素材讲到即可。

回答开头那个质疑：pi 到底算不算工业级？

收官帖的结论：**pi 算工业级，但拿来直接用也不能什么都不管**。"工业级"不体现在代码行数（内核反而极简），而体现在每个设计决定都是**显式取舍且有清晰边界**：错误契约靠类型系统强制、工具流水线防住截断/竞态/并发踩踏、压缩算法防住 25 种 provider 的溢出报错、安全立场诚实到写进 SECURITY.md。而"不能什么都不管"是指：它的沙箱外包给你、它防不了注入、它的默认姿态需要你自己核对——**看懂这些取舍，正是这个系列教的东西**。

一句话带走 Agent 内核 = 一个 while 循环 + 一条错误契约 + 一条流水线 + 一个压缩器 + 一张安全取舍表。110 行能跑，工业级的差距是每一处"真踩过坑才会写"的防御。



主实验：真跑通 step01 mini-agent (含自写协议适配 shim)

背景：本机仅有 z.ai 的 Anthropic 协议端点（OpenAI 兼容端点无余额），而 mini-agent 只会说 OpenAI 的 chat/completions 协议——这正好是 Q3“协议适配层”的活教材，我们顺手写了一个 130 行的零依赖 shim

（`exercise\anthropic-shim.mjs`）：OpenAI 请求 → Anthropic /v1/messages，响应逆向转换，支持流式 SSE 合成，429 自动等 65s 重试。

第 1 步 · 启动协议适配层

`node exercise\anthropic-shim.mjs 8787`（需 ANTHROPIC_BASE_URL / ANTHROPIC_AUTH_TOKEN 环境变量）

目的：给 mini-agent 造一个“假的 OpenAI 端点”。验证：curl 打 `http://localhost:8787/v1/chat/completions`，非流式返回 `{"choices":[{"message":{"content":"OK"}}]}`，流式返回 `data: {...chat.completion.chunk...}` 分帧 + `data: [DONE]` ✓ 两条路径均实测通过

第 2 步 · 在隔离工作区跑 mini-agent

```
cd exercise\live-demo
set LLM_BASE_URL=http://localhost:8787/v1/chat/completions ← 坑：step01 要完整路径（含 /chat/completions）
set DEEPSEEK_API_KEY=shim & set LLM_MODEL=glm-4.6
node ../../repo/steps\01-minimal-loop\agent.mjs "创建文件 poem.txt，内容为三行关于秋天的小诗（中文）。写完后用 run_command 显示它的内容确认。"
```

目的：验证 118 行的循环真的能自主完成“写文件+验证”两段任务。

✓ **实测输出**：agent 自主发起 2 次工具调用——`write_file {"path":"poem.txt","content":"秋风拂过金黄的稻田，..."}` → `run_command {"command":"cat poem.txt"}` → 总结“已完成”。poem.txt 真实落盘（90 字节）✓

第 3 步 · 沙箱拒绝 + 错误回灌验证（实验 B）

```
node agent.mjs "读取 F:\reminder\CLAUDE.md 的内容并总结前100字。"
```

目的：验证 Q2 的核心设计——工作目录外的路径被 `safePath` 拒绝后，错误文本回喂模型会怎样。

✓ **实测输出**：read_file 报错“拒绝访问工作目录之外的路径”→ 错误回灌模型 → 模型改试 run_command 后主动放弃绕过，最终回答：说明沙箱保护是刻意设计、给出“把文件复制进工作区”的替代方案。错误不抛出、还给模型、模型自己修——教科书行为在真机上复现了。

实C-F

四个离线实验：并行顺序 / 压缩 / 安全 / 回归自测（零 API 消耗）

实C · 并行工具顺序 (steps/03)

`node steps/03-tools/demo-order.mjs` → ☒ **实测**：3s/1s/2s 三工具总耗时 3.0s（串行 6s）；完成序 2-3-1、喂回序 1-2-3，"执行是并发的，历史是确定的"（详见 Q6）。

实D · 压缩全过程 (steps/04)

`node steps/04-compaction/demo-compaction.mjs` → ☒ **实测**：19 条消息估算 799 tok；窗口 800、阈值 680 → `shouldCompact=true`；切割点第 7 条；保留 13 条原文 467 tok；压缩后 594 tok、释放 205 tok；文件操作清单从 `tool_calls` 参数提取（`readFiles` / `modifiedFiles`）。

实E · 权限与安全 (steps/05)

`node steps/05-security/demo-security.mjs` → ☒ **实测**：9 条命令 5 条被判危险（`rm -rf` / `sudo` / `curl|sh` / `chmod 777` / `push --force`）；同一条危险命令五种姿态——`off` 放行、`ask` 拒→拦截、`ask` 允→放行、`strict` 直接拦、无人值守 `ask`→`fail-closed` 拦截；`env` 清洗把 `DEEPSEEK_API_KEY` / `GITHUB_TOKEN` / `MY_PASSWORD` 全清掉、保留 `PATH`/`HOME`/`LANG`；非 macOS 上沙箱包裹原样透传 `bash -c`。

实F · 四套离线回归自测

`node steps/0X-*/test.mjs`（ $X=2..5$ ）→ ☒ **实测** $26 + 34 + 44 + 32 = 136$ 条断言全过，0 失败。覆盖：SSE 断流恢复、残缺 JSON 标记失败、rate limit 不误判为 overflow、清掉 `*_API_KEY` 等"真踩过坑"的契约。

环境说明 本机为 Windows：demo-security 的 sandbox-exec 分支按设计原样透传（macOS 专用），test.mjs 亦明确断言"非 macOS 开启也不改变行为"——跨平台行为符合仓库设计，不算缺失。

这个项目给我什么 · 能力清单

① 会读工业级 agent 源码

拿到"行号级"读法：先看调用栈分层（产品层策略/内核层循环/协议层容错），再看错误契约和事件序列闭合这两个不变式。

③ 掌握协议适配层套路

API×Provider 两维拆分；本仓库实验里自写的 OpenAI→Anthropic shim 就是这套思想的 130 行实例，可直接复用在国产端点上。

⑤ 会给长任务 agent 设计压缩

阈值/溢出双路径触发、切割点、结构化摘要模板、文件操作追踪——可搬进任何自研 agent。

② 能从零写一个能干活的 agent

118 行起步：while + 三工具 + safePath；再按 step02-05 逐层加协议适配、工具流水线、压缩、安全——每步可运行可对照。

④ 会选型 coding agent 的安全档位

用 sandbox×approval 两轴评估 pi/codex/grok-build/dsh/Claude Code，先查默认姿态再谈能力，别假设 agent 替你兜底。

⑥ 一套离线可回归的 agent 测试范式

136 条断言示范了怎么测"打不到网络也能测"的 agent 契约（SSE 断流、截断、限流误判、env 清洗）。

产出档案: 本指南 PDF + exercise\ (anthropic-shim.mjs 协议适配层 · live-demo 真跑记录 agent-run.log / agent-sandbox.log / poem.txt) | 上游仓库已克隆至 repo\ (不随总仓库提交) | 知识入库: [[沙箱与审批正交]] [[工具调用三段式流水线]] [[Compaction上下文压缩算法]] [[Provider适配层与错误契约]] 四条新概念, 与 [[Agent循环]] [[AgentHarness智能体挽具]] [[上下文预算与战略压缩]] [[工具调用生命周期]] [[提示注入]] [[ChatModel与消息类型]] 互链。